

Aluno: William Gama Kerpen RA 2903

Professores:

Tutor: Felipe Bonatto

Conteudo: Afonso Brandao e Alan Rodrigo Navia

Disruptive Architectures IOT Big Data e IA

Pipeline IoT para Monitoramento de Temperatura

1. Contextualização do projeto

Ambientes corporativos, residenciais, hospitalares e industriais precisam manter condições térmicas adequadas para garantir conforto, segurança e conservação de ativos. Quando a coleta de temperatura é manual, descentralizada ou não possui histórico organizado, torna-se difícil identificar variações, comparar períodos e agir rapidamente diante de um desvio.

Este projeto propõe um pipeline de dados para leituras de sensores IoT. Os dados de temperatura são recebidos inicialmente por um arquivo CSV, tratados e armazenados em um banco PostgreSQL. Depois, views SQL organizam consultas analíticas recorrentes e um dashboard Streamlit apresenta os dados de forma visual e interativa.

A solução separa as responsabilidades em quatro camadas: Dashboard, Service, Repository e PostgreSQL. O dashboard é responsável pela interação com o usuário; a camada de service concentra regras de tratamento e análise; o repository executa as operações SQL; e o PostgreSQL mantém os dados persistidos. Essa organização facilita manutenção, testes e evolução da aplicação.

2. Passos realizados no desenvolvimento

Antes dos passos demonstrados asseguir, foi lido as orientacoes para entrega do trabalho, o dowload do conjunto de dados "Temperature Readings: IoT Devices" (<https://www.kaggle.com/datasets/atulanandjha/temperature-readings-iot-de-vices>) e a verificação e avaliação do head dos dados para assim avaliar quais os dados obtidos e a formatação dos dados, para que pudesse ter correta

adequação dos dados quando fosse realizado a persistência dos dados com o Postgree.

2.1 Configuração do ambiente

O projeto foi preparado para execução com Python 3.10 ou superior, PostgreSQL 15 ou superior e Docker Desktop. As configurações do banco são definidas por variáveis de ambiente, como ``DB_USER``, ``DB_PASSWORD``, ``DB_NAME``, ``DB_HOST`` e ``DB_PORT``. Dessa forma, o mesmo código pode ser utilizado localmente ou em containers sem alterar os arquivos da aplicação.

2.2 Instalação das dependências

As dependências principais utilizadas foram:

- ``pandas``: leitura, limpeza e transformação dos dados do CSV;
- ``SQLAlchemy``: mapeamento da tabela e comunicação com o banco;
- ``psycopg2-binary``: driver de conexão com PostgreSQL;
- ``streamlit``: construção do dashboard web;
- ``pytest``: testes automatizados;
- ``ruff``: análise de lint e padronização do código.

As dependências da aplicação são instaladas com ``pip install -r requirements.txt``. Para executar testes e lint, são usadas também as dependências de ``requirements-dev.txt``.

2.3 Criação dos containers Docker

O ``Dockerfile`` cria uma imagem Python para executar o dashboard. Ele define o diretório de trabalho, instala as dependências do projeto, copia os arquivos e expõe a porta 8501, utilizada pelo Streamlit.

O arquivo ``docker-compose.yml`` organiza dois serviços. O primeiro é o ``postgres-iot``, responsável pelo PostgreSQL. O segundo é o ``dashboard``, que executa a carga do CSV e inicia o Streamlit. O dashboard só é iniciado após o

healthcheck indicar que o banco está disponível. A solução é executada pelo comando:

```
docker compose up --build
```

Após a inicialização, o dashboard é acessado no navegador em `http://localhost:8501`.

2.4 Inserção dos dados no banco

O arquivo `temperature_readings.csv` possui as colunas de identificação da leitura, sala, data/hora, temperatura e classificação da leitura. Durante a ingestão, a aplicação:

1. padroniza os nomes das colunas;
2. converte data e hora para um formato apropriado;
3. converte a temperatura para valor numérico;
4. remove registros incompletos;
5. remove duplicidades pelo campo `id`;
6. consulta os IDs já existentes antes de inserir novos registros.

Além disso, o campo `out_in` indica a origem da leitura: o valor `In` representa um sensor dentro da sala anônima e `Out` representa um sensor fora da sala. Esse detalhe é importante para comparar o comportamento térmico entre sensores instalados no interior e no exterior do ambiente monitorado. Essa última etapa torna a carga idempotente: executar a ingestão mais de uma vez não duplica eventos no banco. A base analisada possui 97.606 leituras, registradas entre 28/07/2018 e 08/12/2018.

2.5 Criação das views SQL

Depois da criação da tabela `temperature_readings`, o projeto executa o script `create_views.sql`. As views armazenam consultas reutilizáveis no PostgreSQL. Elas não duplicam os dados da tabela: sempre retornam o resultado atualizado da consulta quando são acessadas.

2.6 Construção do dashboard

O dashboard foi desenvolvido com Streamlit e apresenta filtros para visualizar toda a base, o último mês, a última semana ou um dia específico. Também inclui gráficos de média de temperatura por dia, máximas e mínimas diárias e quantidade de leituras por dia.

Para análises específicas, o usuário pode selecionar um dia e consultar os indicadores mínimo, máximo e médio, além da série de temperatura ao longo das horas. Há ainda uma opção para comparar médias de dois meses, semanas ou dias.

2.7 Testes e integração contínua

Foram criados testes unitários para normalização de colunas, filtros temporais e comparação entre períodos. Também foram implementados testes de integração com PostgreSQL real para confirmar a ingestão, imprimir a última leitura persistida e verificar que o dashboard consegue carregar os dados pelo caminho Service → Repository → Banco.

O GitHub Actions executa a integração contínua a cada push ou pull request para a branch `main`. O workflow instala as dependências, inicia um PostgreSQL isolado, executa o lint com Ruff e roda todos os testes com pytest.

3. Explicação das views SQL

3.1 `avg\_temp\_por\_sala`

Esta view agrupa os registros pelo identificador da sala (`room\_id\_id`) e calcula a média de temperatura de cada grupo por meio da função `AVG(temp)`.

O objetivo é permitir a comparação térmica entre ambientes. Em uma implantação com várias salas, ela pode indicar quais locais são mais quentes ou frios e apoiar decisões como ajuste de ar-condicionado, inspeção de isolamento e redistribuição de sensores.

3.2 `leituras\_por\_hora`

Esta view agrupa as medições por hora usando `DATE\_TRUNC('hour', noted\_date)` e contabiliza os registros de cada intervalo por meio de `COUNT(\*)`.

Seu objetivo é analisar a frequência de chegada dos dados. Em um cenário real, a ausência de leituras em uma faixa horária pode indicar falha de sensor, perda de conectividade, indisponibilidade do gateway IoT ou erro no processo de ingestão. Picos de leituras também podem ser investigados para identificar retransmissões ou comportamento inesperado dos dispositivos.

3.3 `temp\_max\_min\_por\_dia`

Esta view agrupa as leituras por dia e retorna a maior e a menor temperatura registrada utilizando `MAX(temp)` e `MIN(temp)`.

O propósito é medir o comportamento térmico diário e sua amplitude. Dias com diferença elevada entre temperatura máxima e mínima podem indicar exposição solar, falha de climatização, abertura frequente de portas, problemas de vedação ou alguma condição operacional que merece investigação.

4. Visualizações do Streamlit

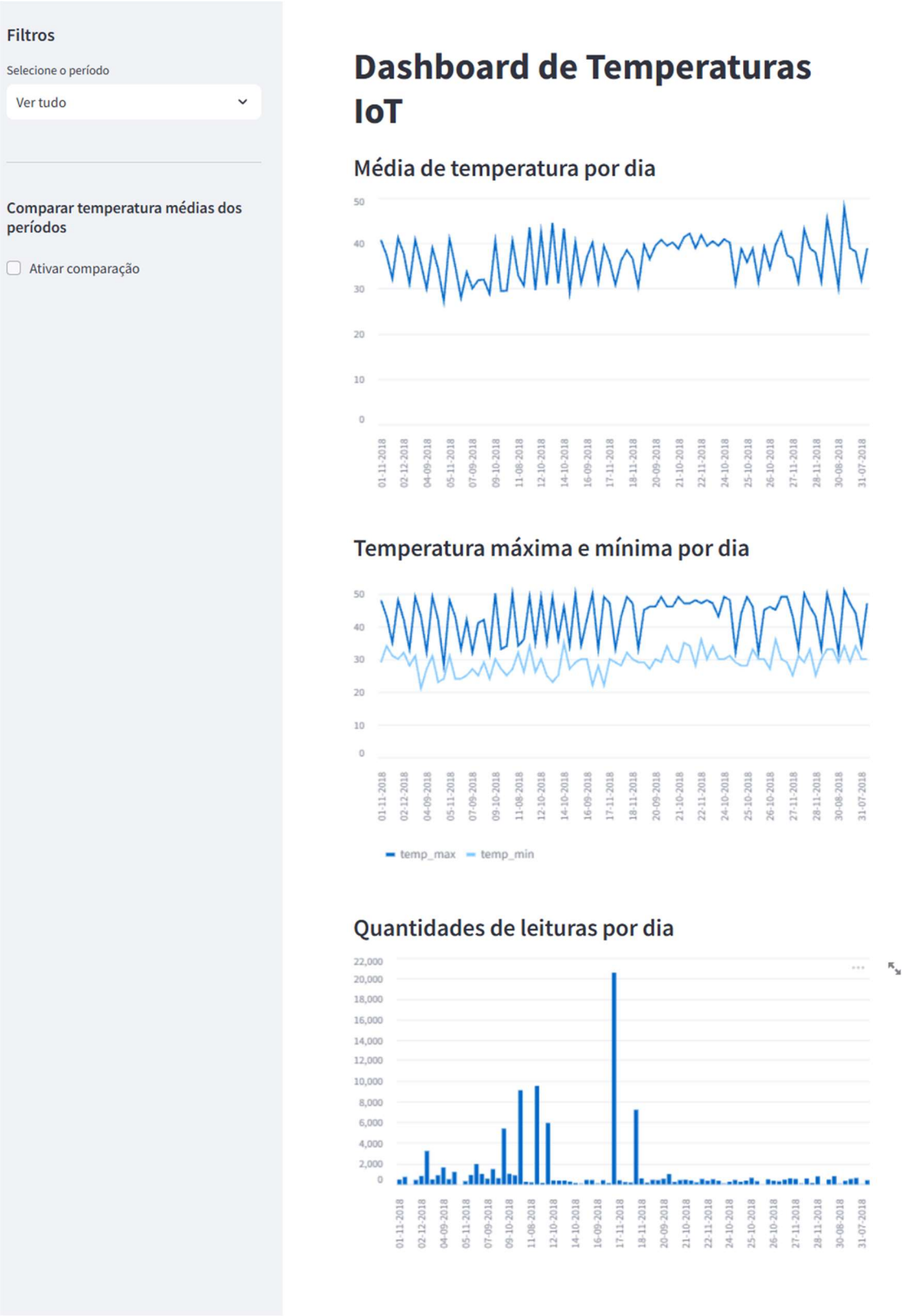


Figura 1 — Dashboard principal com os gráficos de média, temperatura máxima/mínima e quantidade de leituras por dia.

Filtros

Selecione o período

Um dia

Dia específico

2018/12/08

< December 2018 >

Su Mo Tu We Th Fr Sa

1

2 3 4 5 6 7 8

9 10 11 12 13 14 15

16 17 18 19 20 21 22

23 24 25 26 27 28 29

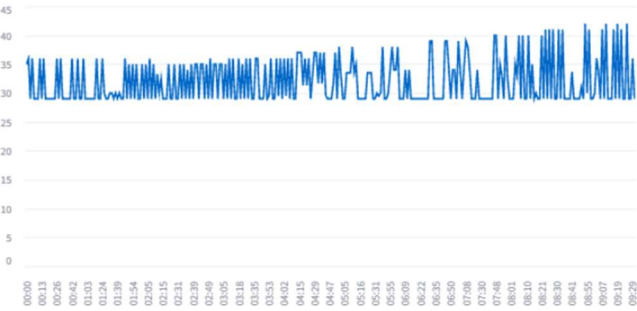
30 31

Dashboard de Temperaturas IoT

Resumo do dia 08-12-2018

Mínima do dia	Máxima do dia	Média do dia
29.00°C	42.00°C	31.98°C

Temperatura ao longo do dia



Média de temperatura por dia



Temperatura máxima e mínima por dia



Quantidades de leituras por dia



Figura 2 — Consulta de um dia específico, com indicadores de mínimo, máximo e média e gráfico intradiário.

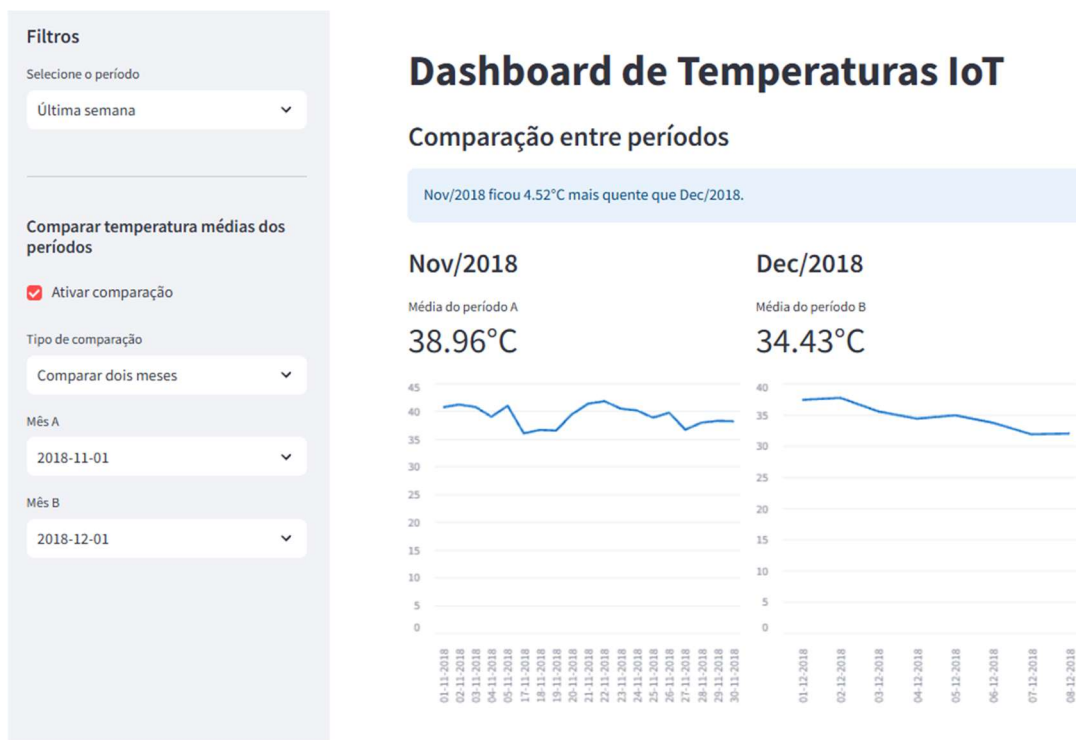


Figura 3 — Comparação das médias de temperatura entre dois períodos selecionados pelo usuário.

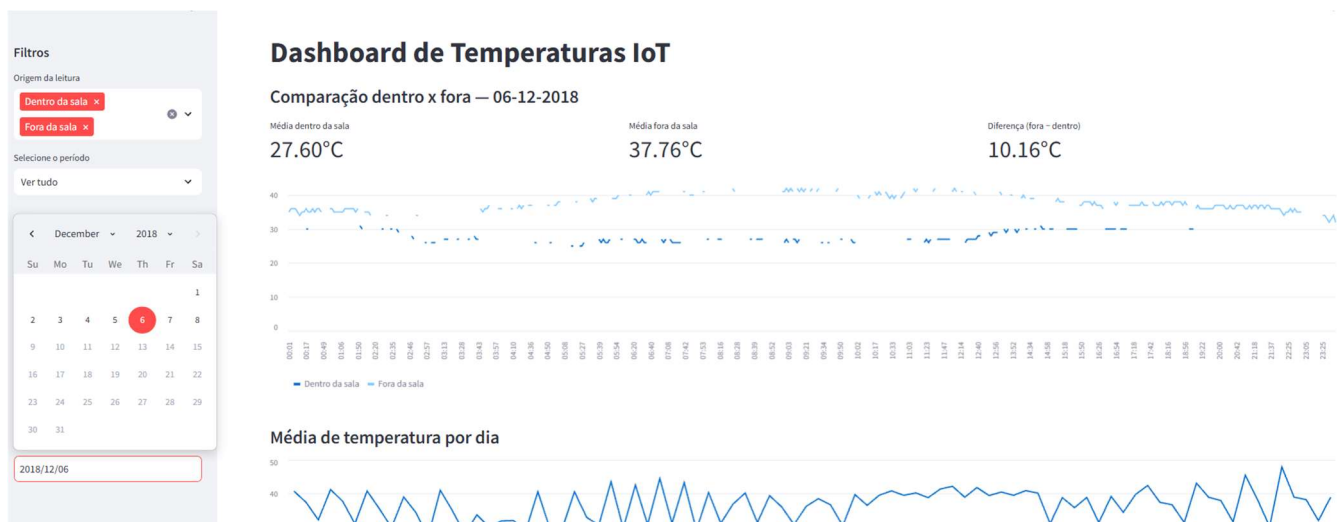


Figura 4 — Comparação das médias de temperatura fora e dentro da sala para um dia selecionado pelo usuário.

5. Principais insights obtidos

A base contém 97.606 leituras e apresenta temperatura média geral de aproximadamente 35,05 °C. O dashboard permite acompanhar como essa média se comporta ao longo dos dias e comparar períodos selecionados pelo usuário, apoiando a identificação de tendências ou mudanças no padrão térmico.

A maior amplitude térmica diária identificada foi de 28 °C em 16/10/2018, quando a temperatura variou de 22 °C a 50 °C. Esse tipo de variação merece atenção em um ambiente real, pois pode representar risco para pessoas, equipamentos, produtos sensíveis ou processos industriais.

Por fim, a base utilizada possui apenas uma sala identificada, `Room Admin`. Embora o pipeline suporte múltiplas salas, a expansão da instrumentação é importante para que comparações entre ambientes se tornem mais representativas.

6. Sugestões de uso prático em um ambiente real

Em uma aplicação real, os sensores poderiam enviar leituras continuamente por MQTT, HTTP ou outro protocolo de IoT, substituindo ou complementando a carga por CSV. O pipeline poderia receber esses eventos, validar os campos e persistir cada leitura no banco quase em tempo real.

Regras de alerta poderiam avisar responsáveis quando a temperatura máxima ultrapassasse um limite, quando a amplitude diária fosse excessiva ou quando um sensor deixasse de enviar dados por determinado período. As notificações poderiam ser enviadas por e-mail, aplicativo de mensagens ou sistema de chamados.

Com um histórico mais longo e dados de contexto, como horário, clima externo e consumo de energia, técnicas de inteligência artificial poderiam identificar anomalias e prever superaquecimento. Isso apoiaria manutenção preventiva de equipamentos de climatização, refrigeração e ventilação.

O mesmo modelo pode ser aplicado a depósitos, hospitais, laboratórios, data centers, supermercados, estufas agrícolas e indústrias. Em todos esses

cenários, o monitoramento contínuo permite reduzir riscos, melhorar eficiência energética e tomar decisões baseadas em dados.

7. Conclusão

Hoje em dia com inúmeras automações, e com a internet das coisas cada vez mais frequente no lares, o número de dados gerados é absurdo, por isso é cada vez mais necessário saber trabalhar com esse dados para poder obter informações através deles, e é isso que o projeto faz. Ele demonstra um fluxo completo de dados IoT, desde a leitura e tratamento de dados até a persistência, análise e visualização. A carga idempotente evita duplicidades, as views SQL organizam consultas recorrentes, o dashboard torna a análise acessível e a integração contínua aumenta a confiabilidade da solução. A arquitetura criada é uma base consistente para evoluir o projeto para uma implantação de monitoramento em tempo real.